

UNIVERSIDAD DE MONTERREY
ESCUELA DE INGENIERÍA Y TECNOLOGÍAS
INTELIGENCIA ARTIFICIAL II

Proyecto Final | Sports Analytics using Deep Learning Models

Alumnos:

Antonio Rivera Aguirre #192238
Jerónimo Alvarez Lozano #586266

Maestro:

Andrés Hernández Gutiérrez

San Pedro Garza García, N.L. 21 de mayo, 2026

Índice

Introduction.....	3
Problem statement.....	4
Methodology.....	5
Results.....	8
Conclusions.....	9
Github.....	10
Honor pledge.....	10
References.....	11

Introduction

En este proyecto se busca aplicar lo aprendido en la materia de Inteligencia Artificial II para desarrollar un modelo capaz de detectar jugadores de fútbol y el balón dentro de un video capturado desde una perspectiva aérea. Para lograrlo, se utilizan conceptos relacionados con deep learning, convolutional neural networks y transfer learning, los cuales permiten identificar objetos dentro de imágenes o secuencias de vídeo mediante bounding boxes.

Deep learning

El “deep learning” se utiliza en este proyecto mediante el modelo YOLOv8, el cual cuenta con múltiples capas que procesan la información visual antes de generar una predicción. A partir de cada frame del video, el modelo analiza patrones, formas y características relevantes para identificar los objetos de interés, en este caso los jugadores y el balón.

Convolutional neural networks

Las “CNN” son un tipo de red neuronal especializada en el procesamiento de imágenes. Estas redes utilizan filtros para extraer características visuales como bordes, texturas, formas y patrones. En el caso de YOLOv8, estas características permiten reconocer objetos dentro del campo de fútbol y generar una bounding box alrededor de cada detección.

Transfer learning:

El “transfer learning” se utiliza al tomar un modelo previamente entrenado y ajustarlo con un custom dataset creado específicamente para este proyecto. En lugar de entrenar un modelo desde cero, se aprovechan los conocimientos previos del modelo base y se reentrena con frames etiquetados del video, usando las clases jugador y pelota. Esto permite adaptar el modelo al contexto específico del video aéreo y mejorar su capacidad para detectar los objetos requeridos.

Modelo YOLOv8

Para el desarrollo del modelo se utilizó YOLOv8 como arquitectura base para realizar transfer learning. YOLOv8 es un modelo de object detection en tiempo real que permite identificar objetos dentro de una imagen mediante bounding boxes, prediciendo la clase del objeto y el nivel de confianza de cada detección. En este proyecto se eligió debido a que cuenta con pretrained weights, documentación clara, compatibilidad con datasets en formato YOLO y una implementación sencilla mediante la librería Ultralytics.

La arquitectura de YOLOv8 se puede dividir en tres componentes principales: backbone, neck y head. El backbone se encarga de extraer características visuales de la imagen, como bordes, texturas y patrones relevantes. Posteriormente, el neck combina información de diferentes escalas para mejorar la representación de objetos pequeños, medianos y grandes. Finalmente, el head realiza las predicciones finales, generando las bounding boxes, la clase detectada y la confianza de cada detección.

En este caso, el modelo fue ajustado con un custom dataset creado a partir de frames del video proporcionado. Las clases utilizadas fueron jugador y pelota, por lo que el entrenamiento permitió adaptar los pesos preentrenados de YOLOv8 al contexto específico del proyecto. Aunque el modelo base ya cuenta con conocimiento previo para detectar personas y objetos, fue necesario aplicar transfer learning debido a que el video tiene una perspectiva aérea, lo cual cambia la apariencia de los jugadores y dificulta especialmente la detección del balón por su tamaño reducido.

Problem statement

El problema principal a resolver en este proyecto consiste en detectar automáticamente a los jugadores de fútbol y la pelota a lo largo de un video capturado desde una perspectiva aérea. Para lograrlo, se busca desarrollar un modelo de object detection capaz de identificar ambas clases mediante bounding boxes en cada frame del video.

El video utilizado presenta una cancha de fútbol vista desde arriba, lo cual representa un reto importante para el modelo. Aunque los jugadores suelen ser más visibles por su tamaño y forma, la pelota es un objeto mucho más pequeño, puede moverse rápidamente y puede confundirse con elementos del fondo. Por esta razón, el modelo debe ser entrenado con un custom dataset que contenga ejemplos representativos de ambos objetos en diferentes posiciones dentro del campo.

Además de detectar jugadores y pelota, el proyecto también requiere utilizar las detecciones para generar visual analytics por frame. Estas analíticas incluyen contar cuántos jugadores se encuentran en el lado izquierdo y derecho de la cancha, cuántos están fuera del campo, cuántos se encuentran dentro de cada área penal, y determinar si la pelota está en el lado

izquierdo, derecho o fuera de la cancha. Estas métricas se calculan usando el centroide de cada bounding box, como se indica en las instrucciones del proyecto.

Por lo tanto, el objetivo del proyecto no solo es entrenar un modelo que detecte correctamente objetos en el video, sino también construir un pipeline completo que procese el video, realice las predicciones, calcule las métricas visuales y genere un video final con las detecciones y la información correspondiente en pantalla.

Methodology

Para el desarrollo de este proyecto se siguió un flujo de trabajo que incluye la creación del dataset, el etiquetado de datos, el entrenamiento del modelo, la validación y prueba del desempeño, el guardado del modelo entrenado y su aplicación sobre el video original para generar las visual analytics.

1. Dataset

El dataset se creó a partir del video proporcionado para el proyecto. El video tiene una resolución de 1280 x 720 píxeles, una duración aproximada de 2 minutos y 8 segundos, 30 frames por segundo y un total de 3,840 frames. Como no era necesario etiquetar todos los frames manualmente, se seleccionó un subconjunto representativo de 580 frames, equivalente aproximadamente al 15% del total del video.

Los frames fueron seleccionados de distintos momentos del video para incluir variaciones en la posición de los jugadores y de la pelota. Esto ayudó a que el modelo pudiera aprender ejemplos del inicio, medio y final de la secuencia, así como diferentes ubicaciones dentro de la cancha

2. Data labelling

El etiquetado se realizó manualmente con la plataforma Roboflow. En cada frame se dibujaron bounding boxes ajustadas alrededor de los objetos visibles, usando dos clases principales: jugador y pelota. Se buscó que las cajas fueran lo más precisas posible, ya que las visual analytics dependen del centroide de cada bounding box para determinar la ubicación de cada objeto dentro o fuera de las regiones definidas.

Después del etiquetado, el dataset se exportó en formato YOLOv8 y se dividió en tres subconjuntos: 70% para entrenamiento, 15% para validación y 15% para prueba. La distribución final fue de 406 frames para train, 85 frames para valid y 85 frames para test.

3. Model selection

Se seleccionó YOLOv8m como modelo base. Esta versión mediana de YOLOv8 ofrece un balance adecuado entre velocidad y precisión: no es tan ligera como las versiones pequeñas, pero puede detectar mejor objetos de menor tamaño, y tampoco es tan pesada como las versiones grandes. Además, permite usar pesos preentrenados, lo que facilita el proceso de transfer learning.

La selección de YOLOv8 también se justificó por su documentación, ejemplos disponibles y compatibilidad con la librería Ultralytics, lo cual facilitó su implementación, entrenamiento, validación y uso para inferencia sobre imágenes y video.

4. Model training

El entrenamiento se realizó mediante fine-tuning, partiendo de pesos preentrenados de YOLOv8m y ajustándolos con el dataset personalizado. Los principales hiperparámetros utilizados fueron 100 épocas, tamaño de imagen de 1280, batch de 4 y patience de 15. Se usó una resolución de 1280 para conservar más detalle en los frames y mejorar la detección de objetos pequeños como la pelota.

El batch se mantuvo en 4 debido a las limitaciones de recursos al entrenar con imágenes de alta resolución. El parámetro “patience” se utilizó para evitar entrenamientos innecesariamente largos si el modelo dejaba de mejorar durante varias épocas.

5. Validation and testing

Durante la validación se revisaron la matriz de confusión, las curvas de pérdida y las métricas principales del modelo. Las pérdidas de entrenamiento y validación, como box_loss, cls_loss y dfl_loss, disminuyeron durante el entrenamiento, lo que indica

que el modelo fue aprendiendo a localizar, clasificar y ajustar mejor las bounding boxes.

Las métricas de precisión, recall, mAP50 y mAP50-95 también aumentaron de forma progresiva. La precisión se mantuvo alta, lo cual indica que cuando el modelo realiza una detección, normalmente corresponde a la clase correcta. El recall mejoró, aunque todavía muestra que algunos objetos, especialmente la pelota, no siempre son detectados.

En el test set, la clase jugador obtuvo un desempeño muy alto, mientras que la clase pelota tuvo un recall menor. Esto confirma que la pelota fue el principal reto del proyecto, debido a su tamaño reducido, movimiento rápido y menor cantidad de ejemplos en comparación con los jugadores.

6. Model saving and loading

Después del entrenamiento, se guardó el mejor modelo obtenido como best.pt y posteriormente se exportó con el nombre YOLOv8-Transfer-ProyectoIA2.pt. Después se cargó nuevamente el archivo guardado para comprobar que el modelo podía inicializarse correctamente y reutilizarse sin necesidad de reentrenar.

7. Inference demonstration and visual analytics

Primero se realizaron predicciones sobre imágenes del test set para observar ejemplos individuales de detección. Posteriormente, se probó el modelo cargado realizando predicciones sobre el video original IA2Video.mp4. Para la inferencia se utilizó un umbral de confianza de 0.50 y un tamaño de imagen de 1280.

El video se procesó frame por frame mediante un ciclo while. En cada frame, el modelo realizó la predicción y después se recorrieron las boxes detectadas para obtener la clase, el centro del objeto y sus coordenadas. Con esta información se contaron los jugadores ubicados en el lado izquierdo, lado derecho, fuera de la cancha y dentro de las áreas penales.

Las líneas de la cancha se dibujaron manualmente usando funciones de OpenCV como cv2.line() y cv2.ellipse(). También se agregó un cuadro informativo usando

cv2.rectangle() y cv2.putText(), donde se muestran los contadores de jugadores y la ubicación estimada de la pelota. Las coordenadas utilizadas fueron obtenidas manualmente abriendo un frame del video en Paint para identificar la posición exacta del cursor.

Una limitación de este enfoque es que las líneas y regiones de la cancha fueron definidas a partir de un solo frame. Debido a que la cámara se mueve ligeramente durante el video, las líneas pueden dejar de coincidir perfectamente con el campo en algunos momentos. Aun así, el método permite generar las visual analytics requeridas para cada frame del video.

Results

En esta tabla se pueden observar las métricas obtenidas del modelo al ponerlo a prueba con el dataset de test:

Tabla 1. Métricas del modelo

Class	Images	Instances	Precisión	Recall	F1-Score
All	85	1102	0.959	0.718	0.821
Jugador	85	1020	0.994	0.996	0.995
Pelota	82	82	0.923	0.439	0.595

En la tabla 1 se pueden observar las tres métricas principales: precisión, recall y F1-score. La clase Jugador tuvo un muy buen desempeño, con una precisión de 0.994, un recall de 0.996 y un F1-score de 0.995. Esto confirma que el modelo logra detectar correctamente a la mayoría de los jugadores y que comete pocos errores en esta clase.

Sin embargo, para la clase Pelota, las métricas fueron más bajas. Se obtuvo una precisión de 0.923, un recall de 0.439 y un F1-score de 0.595. Esto indica que el modelo no logró detectar correctamente muchas de las pelotas, pero las que sí detectó normalmente pertenecían a la clase correcta. La diferencia entre la precisión y el recall evidencia que al modelo todavía se le escaparon varias detecciones de la pelota.

Debido a este desempeño en la clase Pelota, las métricas generales bajan considerablemente. Aun así, se obtuvo una buena precisión general de 0.959, un recall general aceptable de 0.718 y un F1-score general de 0.821, el cual también se considera aceptable. En general, los resultados muestran que el modelo funciona mucho mejor detectando jugadores que detectando la pelota.

Conclusions

Antonio

Después de realizar este proyecto, me queda mucho más claro el uso de transfer learning en aplicaciones de la vida real, como en este caso el análisis deportivo. También me tocó aprender cómo etiquetar frames manualmente con Roboflow y cómo dibujar líneas en los frames usando funciones de cv2, lo cual fue un reto al hacerlo de forma manual y con base en coordenadas obtenidas con Paint.

Haber realizado la investigación en la tarea 8 y la demo del proyecto en la tarea 9 ayudó muchísimo al avance de este proyecto, ya que desde un inicio teníamos seleccionado un modelo. Además, ya sabíamos cómo funcionaba, cuál era su arquitectura, cómo implementarlo y cómo se comportaba con un batch más pequeño del dataset. Por lo tanto, para esta versión final no se comenzó desde cero, sino que ya se tenía una base muy sólida sobre la cual mejorar.

Después de etiquetar muchos frames y probar el modelo con distintos hiperparámetros, obtuvimos un modelo con buen desempeño. La precisión es muy buena, estando arriba de 0.90, pero el recall, aunque está arriba de 0.70, todavía puede mejorar. Probablemente se necesiten más frames en el dataset, ya que este se encuentra muy desbalanceado al tener aproximadamente 12 jugadores por cada pelota.

Realizar este proyecto me dejó mucho aprendizaje sobre los temas vistos en la materia y también me dejó la curiosidad de saber en qué más puedo aplicar esto, ahora que entiendo que no siempre es necesario crear un modelo desde cero.

Jerónimo

El desarrollo de este proyecto me ayudó a reforzar mis aprendizajes adquiridos en clase sobre las redes neuronales convolucionales y transfer learning, y también me permitió ver de manera completa todo el proceso que se lleva a cabo para realizar este tipo de proyectos, desde las fases de investigación de tecnologías y modelos existentes, la preparación del dataset a partir del etiquetado manual, así como la integración de lo que aprendimos en clase y lo que investigamos por nuestra cuenta para este notebook.

Si bien, el modelo final aún tiene problemas en cuanto a la detección correcta de la pelota y en casos muy puntuales los jugadores, estoy satisfecho con cómo quedó el proyecto, esto después de las horas que le dedicamos durante estas semanas a su desarrollo. Algo que me gustaría explorar a futuro a partir de este proyecto es cómo se podría integrar más estadísticas a la parte de visual analytics, como pases completados o incluso porcentaje de posesión por equipo.

Algunas cosas con las que me quedó después de haber finalizado este proyecto son la importancia que tienen las fases de investigación de diferentes herramientas y métodos que se pueden utilizar para el desarrollo de nuevos modelos, así como el haber tenido una experiencia directa de ver cómo se puede desarrollar un modelo de este tipo desde cero.

Github

- Antonio: <https://github.com/antoniorivag/IA2-Proyecto-Final>
- Jeronimo: <https://github.com/jeronimoalvarezl/IA2-Proyecto-Final>

Honor pledge

“Damos nuestra palabra que hemos realizado esta actividad con integridad académica.” -

Antonio, Jerónimo

References

- Administrador. (2020, February 11). Capturar, guardar y leer un video en OpenCV y Python. OMES. <https://omes-va.com/basicvideo/>
- GeeksforGeeks. (2025). Python OpenCV | cv2.imwrite() method. GeeksforGeeks. <https://www.geeksforgeeks.org/python/python-opencv-cv2-imwrite-method/>
- GeeksforGeeks. (2025). Python OpenCV | cv2.putText() method. GeeksforGeeks. <https://www.geeksforgeeks.org/python/python-opencv-cv2-puttext-method/>
- GeeksforGeeks. (2025). Python OpenCV | cv2.rectangle() method. GeeksforGeeks. <https://www.geeksforgeeks.org/python/python-opencv-cv2-rectangle-method/>
- GeeksforGeeks. (2026). cv2.ellipse() method Python OpenCV. GeeksforGeeks. <https://www.geeksforgeeks.org/python/python-opencv-cv2-ellipse-method/>
- GeeksforGeeks. (2026). cv2.line() method Python OpenCV. GeeksforGeeks. <https://www.geeksforgeeks.org/python/python-opencv-cv2-line-method/>
- Módulo IPython.display — Computación Científica con Python. (2022). https://phuijse.github.io/PythonBook/contents/jupyter/ipython_display.html
- moviepy. (2025). PyPI. <https://pypi.org/project/moviepy/>
- Roboflow. (2024). How to Train YOLOv8 Object Detection on a Custom Dataset <https://colab.research.google.com/github/roboflow-ai/notebooks/blob/main/notebooks/train-yolov8-object-detection-on-custom-dataset.ipynb>
- Ultralytics. (n.d.). Reference for ultralytics/engine/results.py | Ultralytics Docs. Ultralytics Docs. https://docs-ultralytics-com.translate.goog/reference/engine/results?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=sge&_x_tr_hist=true

- Videoio (OpenCV 3.4.20 Java documentation). (n.d.).

<https://docs.opencv.org/3.4/javadoc/org/opencv/videoio/Videoio.html>